

Web Crawler

Googlebot · Bingbot · Common Crawl – "queue lo URLs పెట్టి, worker fleet తో లాగేద్దాం" అని అందరూ మొదలుపెడతారు. Fleet పెద్దది చేస్తే వేగం పెరుగుతుందని అనుకుంటారు.

మూడు విరుపులు. మొదటిది – "ఈ URL ఇంతకుముందు చూశామా?" అనే ఒక్క ప్రశ్నకి **8 TB**; bloom filter దాన్ని 5.3 రెట్లు తగ్గిస్తుంది, మరియు **9.8 లక్షల పేజీలు** శాశ్వతంగా పోతాయి. రెండోది – **లక్ష workers పెడితే 34% మాత్రమే** పనిచేస్తాయి. మూడోది – ఒక calendar page మీ బడ్జెట్ లో **99.6%** తినేస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ **PROBLEM** గురించి ఒక అపోహ – మరియు అదే ఈ **DOC** మొత్తం

Crawler అంటే "వేగంగా HTTP requests పంపే system" అని అనిపిస్తుంది. కాబట్టి interview lo చాలామంది వేగం గురించి మాట్లాడతారు – connection pooling, async I/O, worker fleet పరిమాణం.

§1 lo కొలుస్తాం: ఆ భాగం సులభమైనది. సెకనుకి 38,580 pages అంటే 19,290 ఏకకాల fetches – కొన్ని వందల machines. అది ఒక డబ్బు ప్రశ్న, design ప్రశ్న కాదు.

నిజమైన మూడు ప్రశ్నలు వేరే:

1. ఏమి చూశామో ఎలా గుర్తుంచుకోవాలి? (§4 – 8 TB)
2. ఎంత వేగంగా వెళ్ళొచ్చు? (§7 – మీ fleet నిర్ణయించదు)
3. ఎక్కడికి వెళ్ళాలి? (§10 – ఇదే అత్యంత కష్టమైనది)

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: జ్ఞాపకశక్తి

3. Step – చూసిన URLs ని ఒక Set lo
4. మొదటి విరుపు – ఒక్క `has()` కోసం 8 TB
5. Step – Bloom filter, మరియు అది ఏమి ఖర్చు పెడుతుంది

Part 3 – రెండో విరుపు: మర్యాద

6. Step – workers పెంచితే వేగం పెరుగుతుంది
7. రెండో విరుపు – లక్ష workers, 34% వాడకం
8. Step – domain ప్రకారం queue · అసలైన పరిమితి ఏమిటి

Part 4 – మూడో విరుపు: traps

9. Step – ప్రాధాన్యత frontier
10. మూడో విరుపు – బడ్జెట్‌లో 99.6% ఒక calendar మీద
11. Step – మూడు కంచెలు, వాటిలో రెండు పనిచేయవు

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code · 1.6 లక్షల పేజీలు · mutation testing
13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
14. నోటీస్ చెప్పాల్సిన English script
15. Follow-ups – robots.txt, refresh, duplicate content
16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

100 బిలియన్ pages. నెలకి ఒకసారి తిరిగి crawl చేయాలి. ప్రతి page నుంచి links తీసి, వాటిని కూడా crawl చేయాలి.

మొదట సహజమైన భయాన్ని కొలిచి చూద్దాం: **ఇంత వేగం సాధ్యమేనా?**

100 బిలియన్ pages · 30 రోజులకి ఒకసారి refresh

30 రోజులు = 25,92,000 సెకన్లు
→ సెకనుకి 38,580 pages
ఒక్కే page 100 KB → 3.95 GB/s bandwidth
fetch కి 200 ms → 7,716 ఏకకాల fetches కావాలి
fetch కి 500 ms → 19,290 ఏకకాల fetches కావాలి
fetch కి 2000 ms → 77,160 ఏకకాల fetches కావాలి

నిల్వ: 10,240 TB raw HTML

19,290 ఏకకాల fetches. ఒక machine 500 connections నిర్వహించగలిగితే అది 39 machines. Bandwidth 3.95 GB/s – ఒక datacenter కి పెద్ద సంఖ్య కాదు.

అంత వేగం ఒక సమస్య కాదు – మరియు అదే ఈ PROBLEM యొక్క ఉచ్చు

Fetch rate, bandwidth, storage – ఈ మూడూ **డబ్బుతో కొనగలిగేవి**. Machines కలపండి, అవి సరిపోతాయి.

డబ్బుతో కొనలేనివి మూడు, మరియు ఈ doc అవే:

§4 – "ఈ URL చూశామా?" అనే ప్రశ్నకి RAM కావాలి, మరియు RAM తగ్గించే ప్రతి ఉపాయం **పేజీలను పోగొడుతుంది**.

§7 – ఒక website ని సెకనుకి 5 సార్లు కొట్టలేరు. ఆ పరిమితి **మీ fleet పరిమాణంతో సంబంధం లేనిది**.

§10 – web io **అనంతమైన** URLs ఉన్నాయి. మీ బడ్జెట్ పరిమితమైనది. ఏది ముఖ్యమో నిర్ణయించడం ఒక algorithm, ఒక పెద్ద machine కాదు.

అందుకే ఈ doc lo "sharding" గురించి ఒక్క section కూడా లేదు. ఈ problem lo shard key ఎన్నిక కష్టం కాదు – **hash(domain)**, అంతే, ఎందుకంటే మర్యాద నియమాలు domain వారేగా ఉంటాయి కాబట్టి ఒకే domain ఒకే chunk lo ఉండాలి. అది ఒక్క వాక్యంలో ముగిసే నిర్ణయం.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఎందుకు crawl చేస్తున్నాం – search index, archive, training data?	Refresh వ్యూహం పూర్తిగా మారుతుంది. Archive కి ఒకసారి చాలు
ఒక page రెండుసార్లు తెస్తే ఫరవాలేదా?	ఫరవాలేదంటే dedupe approximate కావచ్చు – §5
ఒక page పూర్తిగా మిస్ అయితే?	ఇదే §5 యొక్క నిజమైన ప్రశ్న, మరియు జవాబు సాధారణంగా "ఫరవాలేదు కాదు"
robots.txt పాటించాలా?	అవును అంటే ప్రతి domain కి ఒక అదనపు fetch + cache – §15
ఒక domain ని ఎంత తరచుగా క్రౌల్ చేద్దాం?	ఇదే §7 యొక్క మొత్తం సంఖ్య
JavaScript render చేయాలా?	అవును అంటే fetch ఖర్చు 200 ms నుంచి 2 సెకన్లకి – పై పట్టికలో 10 రెట్లు
ప్రాధాన్యత ఎలా నిర్ణయించాలి?	§9. "అన్నీ సమానం" అంటే §10 lo విరుగుతుంది
Crawler అగి మళ్ళీ మొదలైతే?	Frontier durable గా ఉండాలి – §15

ఒక ప్రశ్న అడిగితే మరు నిజంగా ఒక CRAWLER నడిపారని తెలుస్తుంది

"మీ frontier lo ఎన్ని distinct domains ఉన్నాయి?"

ఎందుకంటే §7 lo కొలుస్తాం – మీ గరిష్ట throughput ని నిర్ణయించేది workers సంఖ్య కాదు, **frontier lo ఉన్న distinct domains సంఖ్య**. 10 domains ఉంటే, లక్ష workers ఉన్నా మీరు సెకనుకి 10 pages మాత్రమే తెస్తారు.

Crawl మొదలైన మొదటి గంటలో ఇది అత్యంత తీవ్రంగా ఉంటుంది – ఎందుకంటే అప్పటికి మీకు కొన్ని seeds మాత్రమే తెలుసు.

Part 2 – మొదటి విరుపు: జ్ఞాపకశక్తి

3. Step – చూసిన URLs ని ఒక Set lo

Crawler యొక్క గుండె ఒక loop:

```
while (frontier.size) {  
  const url = frontier.pop();  
  if (seen.has(url)) continue;    // ఇదే ఈ section  
  seen.add(url);  
  const links = parse(await fetch(url));  
  for (const l of links) frontier.push(l);  
}
```

ఆ `seen` లేకపోతే crawler ఒక వలయంలో ఇరుక్కుపోతుంది – $A \rightarrow B \rightarrow A \rightarrow B$, అనంతంగా. Web lo cycles ఉన్నాయి, మరియు అవి సర్వసాధారణం (ప్రతి page lo "Home" link ఉంటుంది).

కాబట్టి `seen` తప్పనిసరి. ఇప్పుడు ప్రశ్న: **100 బిలియన్ URLs కి ఆ `seen` ఎంత పెద్దది?**

4. మొదటి విరుపు – ఒక్క `has()` కోసం 8 TB

ఒక URL ని ఇంతకుముందు చూశామో లేదో గుర్తుంచుకోవడం

విధానం	ఒక్కే URL కి	100 బిలియన్ URLs కి	తప్పు జవాబు
పూర్తి URL ని నిల్వ	80 bytes	8 TB	లేదు
128-bit hash	16 bytes	1.6 TB	దాదాపు లేదు

ఒక్క **BOOLEAN** ప్రశ్నకి **8 TB RAM**

"ఈ URL చూశామా?" – జవాబు **ఒక్క bit**. కానీ ఆ bit ని **ఏ URL కి** అని గుర్తించడానికే 80 bytes కావాలి.

URL నే కాకుండా దాని 128-bit hash ని నిల్వ చేస్తే 1.6 TB – 5 రెట్లు మెరుగు, మరియు **ఇప్పటికీ ఒక్క machine lo పట్టదు**.

ఇక్కడ ఒక విషయం స్పష్టంగా చెప్పాలి, ఎందుకంటే interview lo చాలామంది దీన్ని తప్పుగా చెబుతారు: **128-bit hash "దాదాపు" తప్పు జవాబు ఇవ్వదు అంటే – collision సంభావ్యత 10^{-18} కంటే తక్కువ**. 100 బిలియన్ URLs కి అది ఆచరణలో సున్నా. కాబట్టి **hash వాడటం ఒక ఉజ్జాయింపు కాదు**; అది ఒక నిజమైన ఆదా.

ఇప్పుడు సహజమైన తర్వాతి అడుగు: **bloom filter**.

5. Step – Bloom filter, మరియు అది ఏమి ఖర్చు పెడుతుంది

Bloom filter ఒక్కో URL కి కొన్ని *bits* మాత్రమే వాడుతుంది – bytes కాదు. బదులుగా అది అప్పుడప్పుడు "చూశాం" అని అబద్ధం చెబుతుంది (false positive). నిజంగా చూడని URL ని "చూశాం" అంటే – ఆ page ఎప్పటికీ crawl కాదు.

Bloom filter – ఒక్కో URL కి ఎన్ని bits?

bits/URL	k	తప్పుడు-సానుకూల రేటు	100 బి URLs కి	ఎన్ని పేజీలు పోతాయి
4	3	14.6892%	50 GB	14,68,91,59,766
8	6	2.1577%	100 GB	2,15,77,14,146
10	7	0.8194%	125 GB	81,93,72,207
16	11	0.0459%	200 GB	4,58,71,073
24	17	0.0010%	300 GB	9,83,858

ఇదొక ఉదాహరణ – సంఖ్యలు రాయకుండా ఈ నిర్ణయం తీసుకోలేము

"0.82% తప్పుడు-సానుకూల రేటు" అనే సంఖ్య చిన్నదిగా అనిపిస్తుంది. 100 బిలియన్ తో గుణిస్తే అది **81 కోట్ల 93 లక్షల పేజీలు** – శాశ్వతంగా, మౌనంగా.

మరియు అవి **యాదృచ్ఛిక** పేజీలు కావు: bloom filter నిండే కొద్దీ రేటు పెరుగుతుంది, కాబట్టి **ఆలస్యంగా కనుగొన్నవి ఎక్కువగా** పోతాయి. అంటే – కొత్త sites.

ఇప్పుడు ఆ ఆదాని నిజంగా పోల్చుదాం. ఇది ముఖ్యం, ఎందుకంటే bloom filter ని "స్పష్టమైన జవాబు" అని అందరూ చెబుతారు:

విధానం	జ్ఞాపకశక్తి	ఆదా	కోల్పోయే పేజీలు
ఖచ్చితమైన 128-bit hash	1,600 GB	–	0
Bloom · 24 bits	300 GB	5.3×	9,83,858
Bloom · 10 bits	125 GB	12.8×	81,93,72,207

ఒక ఋణాత్మక ఫలితం – మరియు దీన్ని నేను చాచడం లేదు

Bloom filter ఈ **problem** కి మంచి ఎంపిక కాదు, మరియు అది నా మొదటి అంచనాకి వ్యతిరేకం.

కారణం: 1,600 GB ని 300 GB కి తగ్గించడం **ఒక machine lo పట్టేలా చేయదు**. రెండూ పంపిణీ చేయాల్సినవే. కాబట్టి మీరు ఒక్క machine కూడా ఆదా చేయలేదు – కేవలం 9.8 లక్షల పేజీలు పోగొట్టుకున్నారు.

Bloom filter ఉపయోగపడేది సరిహద్దు దాటినప్పుడే – అంటే అది మీ data ని "RAM lo పట్టదు" నుంచి "పడుతుంది" కి మారిస్తేనే. ఇక్కడ అది మారదు.

కాబట్టి ఈ doc lo **ఖచ్చితమైన dedupe** వాడతాను: `hash(url)` ని domain ప్రకారం shard చేసిన ఒక పంపిణీ చేయబడిన set. 1.6 TB ని 64 machines మీద పంచితే ఒక్కో దానికి 25 GB – సాధారణం.

BLOOM FILTER ఎప్పుడు సరైనది

ఒక్కో **worker machine** మీద ఒక చిన్న local bloom filter, పంపిణీ చేయబడిన set **ముందు** ఒక cache గా.

ఇక్కడ false positive అంటే "ఈ URL పోయింది" కాదు – "ఒక network call ఆదా అయింది, బహుశా అనవసరంగా". ఎందుకంటే **వెనక ఖచ్చితమైన set ఉంది**.

ఉజ్జాయింపు ఒక cache గా సురక్షితం, ఒక సత్యం గా కాదు.

Part 3 – రెండో విరుపు: మర్యాద

6. Step – workers పెంచితే వేగం పెరుగుతుంది

ఇప్పుడు fleet. సహజమైన model:

$$N \text{ workers} \times \text{ఒక్కో worker సెకనుకి 5 fetches} = 5N \text{ pages/s}$$

10,000 workers → 50,000 pages/s. కావాల్సింది 38,580. సరిపోయింది.

కానీ ఒక నియమం ఉంది, మరియు అది technical కాదు – అది సామాజికమైనది. ఒక website ని మీరు సెకనుకి 100 సార్లు కొడితే, మీరు దాని మీద ఒక DDoS దాడి చేస్తున్నట్టే. కాబట్టి ప్రతి crawler ఒక నియమం పాటిస్తుంది:

ఒక domain ని politenessDelay కంటే వేగంగా కొట్టకూడదు. సాధారణంగా 1 సెకను.

ఇప్పుడు ఆ నియమాన్ని పై లెక్కలో పెడితే ఏమవుతుంది?

7. రెండో విరుపు – లక్ష workers, 34% వాడకం

50 లక్షల URLs ని నిజమైన web లాగా – కొన్ని domains లో లక్షల పేజీలు, చాలా వాటిలో కొన్నే – పంచి కొలిచాను:

50 లక్షల URLs • 2 లక్షల domains

అతి పెద్ద domain : 6,55,485 URLs
p50 : 3
p99 : 184
domains ఉన్నవి : 1,70,633

సగం domains లో 3 పేజీలు. ఒక domain లో 6.5 లక్షలు. ఇప్పుడు మర్యాద నియమంతో:

మర్యాదతో throughput (ఒక్కో domain కి 1 సెకను)

workers	ఆదర్శ throughput	నిజమైన throughput	ఎందుకు
100	500/s	500/s	workers పరిమితి
1000	5,000/s	5,000/s	workers పరిమితి
10000	50,000/s	50,000/s	workers పరిమితి
100000	5,00,000/s	1,70,633/s	మర్యాద పరిమితి

లక్ష WORKERS • 5 లక్షలు రావాలి • 1.7 లక్షలు వచ్చాయి (34%)

66% workers ఏమీ చేయకుండా కూర్చున్నాయి, మరియు వాటికి తీసుకోవాల్సిన URLs frontier lo ఉన్నాయి.

ఎందుకంటే గరిష్ట throughput ని నిర్ణయించేది ఇది:

$\text{max throughput} = \text{distinct domains} \div \text{politenessDelay}$

మీ fleet పరిమాణం ఆ సూత్రంలో లేదు.

ఇక్కడే ఒక ఋణాత్మక ఫలితాన్ని కూడా చెప్పాలి – ఎందుకంటే నా మొదటి ముగింపు అతిశయోక్తి. నిజమైన web lo 20 కోట్ల domains ఉన్నాయి, కాబట్టి ఆ సూత్రం ప్రకారం పరిమితి సెకనుకి 20 కోట్లు – మనకి కావాల్సిన 38,580 కంటే చాలా ఎక్కువ. **అంటే మొత్తం throughput ఒక సమస్య కాదు.**

సమస్య **పంపిణీ**, మరియు అది రెండు చోట్ల కొడుతుంది:

అతి పెద్ద domain: 6,55,485 URLs • సెకనుకి ఒకటి

→ 7.6 రోజులు • ఆ ఒక్క site కోసం

(మీ దగ్గర లక్ష workers ఉన్నా – ఆ domain కి *ఒక్కటే* వాడగలరు)

రెండు నిజమైన పరిణామాలు

1. తోక (tail). ఒక పెద్ద site ని పూర్తి చేయడానికి **7.6 రోజులు**, మరియు ఆ సమయాన్ని machines కలపడం ద్వారా తగ్గించలేరు. మీ "30 రోజుల refresh" హామీ ఆ ఒక్క domain చేతిలో ఉంది.

2. మొదలు. Crawl మొదలైనప్పుడు మీకు 10 seed URLs మాత్రమే తెలుసు – అంటే 10 domains – అంటే **సెకనుకి 10 pages**, మీ fleet ఎంత పెద్దదైనా. Frontier lo domain వైవిధ్యం పెరిగే కొద్దీనే throughput పెరుగుతుంది.

8. Step – domain ప్రకారం queue • అసలైన పరిమితి ఏమిటి

పరిష్కారం ఒకే queue కాదు – **domain వారీగా విడి queues**, మరియు ప్రతి domain కి "ఎప్పుడు మళ్ళీ కొట్టొచ్చు" అనే ఒక timestamp:

```
#lastFetch = new Map(); // domain → ఎప్పుడు చివరిసారి కొట్టాం

const last = this.#lastFetch.get(u.dom);
if (last !== undefined && now - last < this.politenessMs) {
  deferred.push(u); // వదిలేయకూడదు – వాయిదా వేయాలి
  continue;
}
```

ఆ `deferred.push(u)` అనే ఒక్క పంక్తి – అది వదిలేస్తే ఏమవుతుందో **\$12 lo కొలుస్తాం**, మరియు ఫలితం ఆశ్చర్యకరమైనది.

మర్యాద ఒక పరిమితి కాదు – అది ఒక SHARD KEY

మర్యాద నియమాలు domain వారీగా ఉంటాయి కాబట్టి, ఒక domain యొక్క అన్ని URLs ఒక machine మీద ఉండాలి. లేకపోతే ఆ "చివరిసారి ఎప్పుడు కొట్టాం" అనే విషయాన్ని machines మధ్య సమన్వయం చేయాలి – ప్రతి fetch కి ఒక network round-trip.

`shard = hash(domain) % N`

అప్పుడు ఆ timestamp పూర్తిగా స్థానికమైనది, మరియు ఏ సమన్వయమూ అవసరం లేదు. ఈ problem lo sharding నిర్ణయం అంత సులభం – ఎందుకంటే domain మీద shard చేయడం తప్ప వేరే దారి లేదు.

Part 4 – మూడో విరుపు: traps

9. Step – ప్రాధాన్యత frontier

Web అనంతమైనది. మీ బడ్జెట్ పరిమితమైనది. కాబట్టి frontier ఒక FIFO queue కాకూడదు – అది ఒక **ప్రాధాన్యత queue** కావాలి, ముఖ్యమైన పేజీలు ముందు.

ప్రాధాన్యత ఎక్కడి నుంచి వస్తుంది? సాధారణ సూత్రం: **ముఖ్యమైన పేజీ నుంచి వచ్చిన link ముఖ్యమైనది:**

```
child.priority = parent.priority * 0.85;
```

(ఆ 0.85 PageRank యొక్క damping factor – ఒకే ఆలోచన.) Seeds కి అత్యధిక ప్రాధాన్యత, ప్రతి hop కి కొంచెం తగ్గుతుంది.

ఇది సహేతుకంగా అనిపిస్తుంది. **ఇప్పుడు దాన్ని ఒక trap మీద నడుపుదాం.**

10. మూడో విరుపు – బడ్జెట్లో 99.6% ఒక calendar మీద

Trap అంటే అనంతమైన URLs ఉత్పత్తి చేసే ఒక site. అత్యంత సాధారణమైనది ఒక calendar:

```
/calendar?month=2026-09 → to "తర్వాతి నెల" link → /calendar?month=2026-10  
→ అది → /calendar?month=2026-11 → ...
```

ఇది దాడి కాదు – ఇది ఒక సాధారణ calendar widget. అది కేవలం అనంతంగా ఉంటుంది. Session ids, sort parameters, filter combinations – అన్నీ అదే ఆకారం.

5 లక్షల పేజీల బడ్జెట్, 20,000 మంచి domains, 3 traps. **ఒక trap ని 3వ అతి ముఖ్యమైన site మీద పెట్టాను, ఎందుకంటే నిజ జీవితంలో traps తక్కువ నాణ్యత sites మీద ఉండవు – అవి పెద్ద sites మీద ఉంటాయి.**

5 లక్షల పేజీల బడ్జెట్ · 20,000 మంచి domains · 3 trap domains
(ఒక trap 3వ అతి ముఖ్యమైన site మీద ఉంది · ప్రాధాన్యత frontier)

విధానం	trap పేజీలు		బడ్జెట్లో %	చేరిన మంచి domains
పరిమితి లేకుండా	4,98,062	99.6%	3 (0%)	
domain కి 5,000 పరిమితి	10,000	2.0%	333 (2%)	
+ లోతు పరిమితి 12	10,000	2.0%	333 (2%)	
+ domain ప్రాధాన్యత తగ్గింపు	543	0.1%	20,000 (100%)	

మొదటి వరుస - 99.6% బడ్జెట్ మూడు DOMAINS మీద - 20,000 LO 3 చేరాం

ప్రాధాన్యత frontier సరిగ్గానే పనిచేసింది. అదే సమస్య.

Trap ఒక ముఖ్యమైన site మీద ఉంది → దాని పేజీలకి అధిక ప్రాధాన్యత → అవి ముందు వస్తాయి → ప్రతి trap page ఇంకీ trap page ని అధిక ప్రాధాన్యతతో ఉత్పత్తి చేస్తుంది → **frontier శాశ్వతంగా ఆ ఒక్క site తో నిండిపోతుంది.**

ఇది ఒక bug కాదు. ఇది ప్రాధాన్యత నియమాన్ని ఖచ్చితంగా పాటించడం.

11. Step – మూడు కంచెలు, వాటిలో రెండు పనిచేయవు

పై పట్టికలోని మిగతా మూడు వరుసలే అసలు కథ. మూడు స్పష్టమైన పరిష్కారాలను ఒకదాని తర్వాత ఒకటి కలిపాను.

కంచె 1 – ఒక్క domain కి గరిష్ట పేజీలు (5,000).

```
if (n >= this.perDomainCap) { this.stats.skipCap++; continue; }
```

Trap పేజీలు 4,98,062 నుంచి **10,000 కి** పడ్డాయి – 99.6% నుంచి 2.0%. విజయం, అనిపిస్తుంది.

కానీ చేరిన మంచి DOMAINS: 3 → **333**. అంటే 20,000 LO **2%**

Trap ని ఆపాం, మరియు **crawl ఇప్పటికీ విఫలమైంది.**

ఎందుకంటే trap ఆగాక కూడా ప్రాధాన్యత క్రమం మారలేదు – frontier ఇప్పుడు తర్వాతి అత్యధిక ప్రాధాన్యత గల కొన్ని domains తో నిండిపోయింది, ఒక్కొక్కటి 5,000 పేజీలు. $5 \text{ లక్షలు} \div 5,000 = 100 \text{ domains}$, ఆపై కొన్ని చిన్నవి.

నేను trap ని పరిష్కరించాను. అసలు సమస్యని కాదు.

కంచె 2 – లోతు పరిమితి (12 hops). Calendar trap ఒక గొలుసు కాబట్టి లోతు పరిమితి దాన్ని కత్తిరించాలి, అనిపిస్తుంది.

+ లోతు పరిమితి 12		10,000		2.0%		333 (2%)
-------------------	--	--------	--	------	--	----------

ఒక్క సంఖ్య కూడా మారలేదు. ఎందుకంటే domain పరిమితి ఇప్పటికీ 5,000 దగ్గర ఆపేసింది, మరియు trap ఆ 5,000 ని **12 hops లోపే** ఉత్పత్తి చేస్తుంది (ప్రతి page నుంచి 4 links అంటే $4^{12} \gg 5,000$). లోతు పరిమితి తర్వాత ఉన్న కంచె – అది ఎప్పటికీ చేరదు.

ఇది ఒక సాధారణ INTERVIEW తప్పు, మరియు ఇక్కడ అది కొలవబడింది

"లోతు పరిమితి పెడతాను" అనేది సరైన జవాబులా అనిపిస్తుంది, మరియు ఈ సందర్భంలో అది పూర్తిగా నిరుపయోగం – ఎందుకంటే traps లోతైనవి కాదు, వెడల్పైనవి.

లోతు పరిమితి ఇప్పటికీ ఉంచాను, కానీ వేరే కారణానికి: పొడైన relative links (/a/a/a/a/...) నుంచి రక్షణ. దాన్ని trap రక్షణ అని చెప్పడం మాత్రం అబద్ధం.

కంచె 3 – ఒక domain నుంచి ఎక్కువ తీసుకున్న కొద్దీ దాని ప్రాధాన్యత తగ్గించడం.

```
complete(u, links = []) {
  const n = this.#count.get(u.dom) || 1;
  const damp = this.damping ? 1 / (1 + n) : 1; // ← ఇదే
  for (const l of links) this.#add(l, u.depth + 1, u.p * 0.85 * damp);
}
```

ఆ $1/(1+n)$ – ఒక domain నుంచి 100 పేజీలు తీసుకున్నాక, ఆ domain యొక్క కొత్త links యొక్క ప్రాధాన్యత 101 రెట్లు తగ్గుతుంది.

+ domain ప్రాధాన్యత తగ్గింపు	543	0.1%	20,000 (100%)
------------------------------	-----	------	---------------

TRAP పేజీలు 0.1% • చేరిన DOMAINS 20,000 (100%)

ఒకే మార్పు రెండు సమస్యలనూ పరిష్కరించింది – ఎందుకంటే రెండూ ఒకే సమస్య: frontier ఒక్క వనరుతో నిండిపోవడం.

Domain పరిమితి ఒక గోడ – "5,000 తర్వాత ఆపు". అది trap ని ఆపుతుంది, కానీ మొదటి 5,000 ఇప్పటికీ ఇతరుల కంటే ముందు వస్తాయి.

ప్రాధాన్యత తగ్గింపు ఒక వాలు – "ఎక్కువ తీసుకున్న కొద్దీ తక్కువ ఆసక్తి". ఇది crawl ని సహజంగా వెడల్పుగా చేస్తుంది.

గోడలు నష్టాన్ని పరిమితం చేస్తాయి. వాలులు ప్రవర్తనని మారుస్తాయి.

మూడు కంచెలూ ఉంచుతున్నాను, కానీ వాటి పాత్రలు వేరు, మరియు ఆ తేడా చెప్పగలగడమే ఈ section యొక్క సారాంశం:

కంచె	ఏమి చేస్తుంది	trap ని ఆపుతుందా
Domain పరిమితి	అత్యంత చెడ్డ సందర్భంలో నష్టానికి ఒక పైకప్పు	ఆపుతుంది, కానీ coverage 2%
లోతు పరిమితి	పొడైన relative links నుంచి రక్షణ	లేదు – ఒక్క సంఖ్య మారలేదు
ప్రాధాన్యత తగ్గింపు	crawl ని వెడల్పుగా చేస్తుంది	అవును – 0.1%, coverage 100%

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code • 1.6 లక్షల పేజీలు • mutation testing

ఇప్పుడు మూడు విరుపుల పరిష్కారాలూ ఒకే చోట:

```
class Crawler {
  #seen = new Set(); // §5 – ఖచ్చితమైన dedupe (bloom కాదు)
  #frontier = new Heap(); // §9 – ప్రాధాన్యత queue
  #lastFetch = new Map(); // §8 – domain → చివరిసారి ఎప్పుడు
  #count = new Map(); // §11 – domain → ఎన్ని తీసుకున్నాం

  #add(url, depth, p) {
    const dom = domainOf(url);
    if (!dom) { this.stats.badUrl++; return false; }
    if (this.#seen.has(url)) { this.stats.dupe++; return false; }
    this.#seen.add(url); // *enqueue* అప్పుడే గుర్తుపెట్టడం
    this.#frontier.push({ url, dom, depth, p });
    this.stats.enqueued++;
    return true;
  }

  next() {
    const now = this.#clock();
    const deferred = [];
    let out = null;
    while (this.#frontier.size) {
      const u = this.#frontier.pop();
      const n = this.#count.get(u.dom) || 0;
      if (n >= this.perDomainCap) { this.stats.skipCap++; continue; } // §11
      if (u.depth > this.maxDepth) { this.stats.skipDepth++; continue; } // §11
      const last = this.#lastFetch.get(u.dom);
      if (last !== undefined && now - last < this.politenessMs) { // §8
        this.stats.skipPolite++; deferred.push(u); continue;
      }
      out = u; break;
    }
    for (const d of deferred) this.#frontier.push(d); // వాయిదా – వదిలేయడం కాదు
    if (!out) return null;
    this.#lastFetch.set(out.dom, now);
    this.#count.set(out.dom, (this.#count.get(out.dom) || 0) + 1);
    this.stats.fetched++;
    return out;
  }

  complete(u, links = []) {
    const n = this.#count.get(u.dom) || 1;
    const damp = this.damping ? 1 / (1 + n) : 1; // §11
    for (const l of links) this.#add(l, u.depth + 1, u.p * 0.85 * damp);
  }
}
```

```
}  
}
```

ఒక్క సూక్ష్మమైన విషయాన్ని గమనించండి: `#seen.add(url) enqueue అప్పుడు` జరుగుతుంది, `fetch` అప్పుడు కాదు. ఎందుకంటే ఒకే URL frontier lo పది చోట్ల ఉండొచ్చు (పది పేజీలు దానికి link ఇచ్చాయి). Fetch అప్పుడు గుర్తుపెడితే ఆ పది కాపీలూ frontier lo కూర్చుంటాయి.

ఇప్పుడు 1,000 యాదృచ్ఛిక crawls – యాదృచ్ఛిక మర్యాద ఆలస్యాలు, పరిమితులు, traps, పొడైన URLs, నకిలీ links:

```
1,000 యాదృచ్ఛిక ప్రయోగాలు · 1,60,842 పేజీలు · 3,05,278 URLs queue lo  
నియమ ఉల్లంఘనలు: 0
```

ఏ దారులు నడిచాయి:

seeded	7,491
enqueued	3,05,278
dupe	13,038
fetches	1,60,842
skipCap	97,915
skipDepth	46,521
skipPolite	51,66,943
badUrl	7,959

ఆ రెండో పట్టిక ఎప్పుడూ కావాలి – **ప్రతి కంచె నిజంగా తాకబడిందని** అది నిరూపిస్తుంది. `skipCap 97,915`, `skipDepth 46,521`, `badUrl 7,959` – ఏ దారే పరీక్షించకుండా మిగలలేదు.

"0 ఉల్లంఘనలు" అనేది ఒక **పరీక్ష విఫలం కాగలదని** నిరూపించాకే అర్థవంతమైనది:

```
==== mutation testing ====
```

మార్పు లేని code	→	0/1000 విఫలం
మర్యాద పరీక్ష తీసేస్తే	→	299/300 విఫలం
ఉదా: d0.com: 319 ms lo రెండుసార్లు (పరిమితి 1454)		
dedupe ని enqueue కాకుండా fetch అప్పుడు చేస్తే	→	300/300 విఫలం
ఉదా: seen 0 < fetched 60		
domain పరిమితి తీసేస్తే	→	271/300 విఫలం
ఉదా: d0.com: 6 > పరిమితి 5		
లోతు పరిమితి తీసేస్తే	→	210/300 విఫలం
ఉదా: http://d3.com/45-1-573990: లోతు 5 > 4		
వాయిదా వేసినవాటిని frontier కి తిరిగి పెట్టకపోతే	→	0/300 విఫలం

చివరి వరుస ముఖ్యమైనది. `deferred` ని తిరిగి frontier lo పెట్టే పంక్తిని తీసేశాను – అంటే మర్యాద కారణంగా వాయిదా వేసిన ప్రతి URL **శాశ్వతంగా పోతుంది**. నా అయిదు నియమాల్లో **ఒక్కటి దాన్ని పట్టుకోలేదు**.

అవి పట్టుకోలేవు, ఎందుకంటే అయిదు "ఏదో తప్పు జరగకూడదు" అనే ఆకారంలో ఉన్నాయి: ఒకే URL రెండుసార్లు కాదు, మర్యాద ఉల్లంఘన కాదు, పరిమితి దాటడం కాదు. URLs ని వదిలేస్తే **ఏదో తప్పు జరగదు** – తక్కువ జరుగుతుంది.

```
20,000 అవకాశాలు · 40 domains · 1 s మర్యాద ఆలస్యం
```

వాయిదాలు వెనక్కి పెడితే → 20,000 పేజీలు · 40 domains · frontier lo 40,040 మిగిలాయి
పెట్టకపోతే → 13,690 పేజీలు · 40 domains · frontier lo 0 మిగిలాయి

కోల్పోయినవి: 6,310 పేజీలు (31.6%) – ఒక్క నియమ ఉల్లంఘన లేకుండా

31.6% PAGES మౌనంగా పోయాయి · అయిదు నియమాల్లా నెగ్గాయి

ఆ "frontier lo 0 మిగిలాయి" అనే సంఖ్య అసలు సాక్ష్యం. ఆరోగ్యకరమైన crawler యొక్క frontier ఎప్పుడూ పెరుగుతూ ఉండాలి – ప్రతి page కొత్త links తెస్తుంది కాబట్టి. Frontier ఖాళీ అయిందంటే crawler "ముగించింది" కాదు, అది పనిని పోగొట్టుకుంది.

ఇదే HLD Deep 10 lo కూడా వచ్చింది, అదే ఆకారంలో. కాబట్టి ఇది యాదృచ్ఛికం కాదు – ఇది ఒక నమూనా:

"ఏదీ తప్పు జరగలేదు" నియమాలు ఎప్పుడూ ఒక "ఏదీ జరగాలి" నియమంతో జతగా ఉండాలి.

ఆ జత నియమం ఇక్కడ: మర్యాద ఆలస్యం కంటే ఎక్కువ సమయం నడిపితే, వాయిదా వేసిన ప్రతి URL తప్పక తిరిగి రావాలి. దాన్ని కొలిచే సరళమైన రూపం పైదే – `frontier.size` సున్నాకి పడకూడదు.

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-5	Requirements + స్కోప్	100 బి pages, 30 రీజల refresh
5-8	వేగం సమస్య కాదని చూపించడం	38,580/s · 19,290 ఏకకాల fetches · "ఇది డబ్బు ప్రశ్న"
8-14	Dedupe → 8 TB → bloom → ఎందుకు వద్దు	8 TB · 1.6 TB · 24 bits = 300 GB కానీ 9.8 లక్షలు పోతాయి
14-22	మర్యాద → throughput సూత్రం → తేక	<code>domains ÷ delay</code> · అతి పెద్ద site కి 7.6 రీజలు
22-32	Traps – ఇదే ముఖ్యమైన భాగం	99.6% బడ్జెట్ · domain పరిమితి తర్వాత coverage 2%
32-38	ప్రాధాన్యత తగ్గింపు · గోడ vs వాలు	0.1% trap, 100% coverage
38-45	Sharding, durability, robots.txt, follow-ups	<code>hash(domain)</code> · §15

ఈ PROBLEM LO మమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "domain కి ఒక పరిమితి పెడతాను" అని చెప్పి trap ప్రశ్నని ముగిస్తారు.

మీరు ఒక అడుగు ముందుకి వెళ్తుండి: "ఆ పరిమితి trap ని ఆపుతుంది, కానీ crawl ఇప్పటికీ విఫలమవుతుంది – నేను కొలిచినప్పుడు coverage 2% మాత్రమే. ఎందుకంటే పరిమితి ఒక గోడ, మరియు frontier ఇప్పుడు తర్వాతి 100 domains తో నిండుతుంది. కావాల్సింది ఒక వాలు – తీసుకున్న కొద్దీ ప్రాధాన్యత తగ్గాలి."

ఇది ఒక నిర్దిష్టమైన, కొలవబడిన, తప్పుని సరిదిద్దుకున్న జవాబు – మరియు అది మీరు దీన్ని నిజంగా నడిపారని చూపిస్తుంది.

14. నోటీతో చెప్పాల్సిన English script

వేగం గురించి:

"Let me first check whether throughput is even the hard part. A hundred billion pages on a thirty-day refresh is about thirty-nine thousand pages per second. At half a second per fetch that's nineteen thousand concurrent connections – maybe forty machines. So fetch rate is a budget question, not a design question. The hard parts are memory, politeness, and deciding where to go."

Dedupe గురించి:

"Seen-set at a hundred billion URLs is eight terabytes if I store URLs, 1.6 if I store 128-bit hashes. The usual answer here is a bloom filter, but I want to be careful: at twenty-four bits per URL it's three hundred gigabytes – a 5.3× saving – and it silently drops about a million pages. Both numbers still need a distributed set, so I haven't saved a single machine. I'd keep the exact set as the source of truth, sharded by domain, and use a small bloom filter per worker as a cache in front of it. Approximation is safe as a cache, not as a truth."

మర్యాద గురించి:

"Politeness caps me at one request per second per domain, so my ceiling is distinct-domains divided by delay – my fleet size isn't in that formula. Across the whole web that ceiling is huge, so it's not the bottleneck. What does bite is the distribution: in my simulation the largest domain had six hundred fifty thousand URLs, which is 7.6 days at one per second, and no amount of hardware shortens it. The same effect hits at startup, when I only know ten domains."

Traps గురించి:

"Then the interesting failure. I ran a priority frontier against a calendar trap sitting on the third most important site, with a five-hundred-thousand page budget. It spent 99.6% of the budget on three domains and reached three good domains out of twenty thousand. A per-domain cap of five thousand fixed the trap – but coverage was still two percent, because the cap is a wall and the frontier just refilled with the next hundred domains. A depth limit changed nothing at all, because traps are wide, not deep. What actually worked was damping priority by how much I'd already taken from that domain – traps down to 0.1%, coverage to a hundred percent. Walls bound the damage; slopes change the behaviour."

15. Follow-ups – robots.txt, refresh, duplicate content

"robots.txt ఎలా నిర్వహిస్తారు?"

ప్రతి domain కి ఒక అదనపు fetch, ఆపై cache (సాధారణంగా 24 గంటలు). ముఖ్యమైన విషయం: అది మీ shard నిర్ణయాన్ని **ధృవీకరిస్తుంది** – robots.txt domain వారీగా ఉంటుంది, `#lastFetch` వారీగా ఉంటుంది, `#count` వారీగా ఉంటుంది. **మూడూ ఒకే చోట ఉండాలి**. robots.txt lo `Crawl-delay` ఉంటే అది మీ `politenessMs` ని ఆ domain కి overwrite చేస్తుంది.

"ఒక page ని ఎప్పుడు మళ్ళీ crawl చేయాలి?"

ఇదే నిజమైన production ప్రశ్న, మరియు "30 రోజులకి ఒకసారి అందరికీ" అనేది తప్పు జవాబు – ఒక news homepage నిమిషానికి మారుతుంది, ఒక 2009 blog post ఎప్పటికీ మారదు. ఆచరణలో: ప్రతి page కి గత మార్పుల ఆధారంగా ఒక **అంచనా వేసిన మార్పు రేటు**, మరియు refresh వ్యవధి దానికి అనులోమానుపాతంలో. `Last-Modified` మరియు `ETag` తో conditional GET – అది మారకపోతే 304, bandwidth ఆదా, కానీ **మర్యాద slot మాత్రం ఖర్చవుతుంది** (అదే §7 యొక్క పరిమితి).

"ఒకే విషయం వేరే URLs lo ఉంటే?"

URL dedupe (§5) ఇక్కడ సహాయపడదు – URLs వేరు, content ఒకటే. దీనికి **content ఆధారిత dedupe** కావాలి: పేజీ text యొక్క SimHash లేదా MinHash, ఆపై దగ్గరి పొరుగులను వెతకడం. ఇది §5 కంటే చాలా ఖరీదైనది, మరియు ఇక్కడ **ఉజ్జాయింపు సరైనది** – ఎందుకంటే "ఇవి రెండూ దాదాపు ఒకటే" అనేది అసలే ఒక ఉజ్జాయింపు ప్రశ్న.

"Crawler ఆగిపోతే?"

Frontier durable గా ఉండాలి – కేవలం memory heap కాదు. ఆచరణలో ఒక priority queue ని disk మీద (ఒక LSM store lo, key = priority), మరియు `#seen` ఇప్పటికే పంపిణీ చేయబడినది. మళ్ళీ మొదలైనప్పుడు `#lastFetch` పోతుంది – అది **సురక్షితమైన** నష్టం, ఎందుకంటే అది మిమ్మల్ని అవసరం కంటే ఎక్కువ మర్యాదగా చేస్తుంది, తక్కువ కాదు.

"హానికరమైన sites?"

§11 యొక్క మూడు కంచెలూ ఇక్కడ రక్షణగా పనిచేస్తాయి, కానీ అదనంగా: ప్రతిస్పందన పరిమాణానికి ఒక పరిమితి (zip bomb), redirect గొలుసులకి పరిమితి, మరియు ప్రతి fetch కి timeout. **ఒక పెద్ద పరిశీలన** – ఈ మూడూ trap రక్షణ కాదు, అవి వనరుల రక్షణ. §11 యొక్క పాఠం మళ్ళీ: ప్రతి కంచె ఏమి చేస్తుందో ఖచ్చితంగా చెప్పగలగాలి.

16. ఏమి నేర్చుకున్నాం

1. సులభమైన భాగాన్ని ముందే కొలిచి వక్రన పెట్టాలి. Fetch rate 38,580/s – 40 machines. అది డబ్బు ప్రశ్న. ఆ లెక్క చేయడమే మిగతా 40 నిమిషాలు ఎక్కడ ఖర్చు పెట్టాలో చెబుతుంది.
2. ఉజ్జాయింపు ఒక cache గా సురక్షితం, ఒక సత్యం గా కాదు. Bloom filter 5.3 రెట్లు ఆదా చేసింది మరియు ఒక్క machine కూడా ఆదా చేయలేదు – ఎందుకంటే అది ఏ సరిహద్దున దాటలేదు. ఖర్చు: 9,83,858 పేజీలు.
3. మే fleet పరిమాణం throughput సూత్రంలో లేదు. $domains \div delay$. మరియు ఒక్క పెద్ద site కి 7.6 రోజులు – machines కలిపినా తగ్గదు.
4. ప్రాధాన్యత నియమాన్ని ఖచ్చితంగా పాటిస్తే crawl 99.6% బడ్జెట్ ని మూడు domains మీద ఖర్చు పెడుతుంది. ఇది bug కాదు, ఇది నియమం.
5. సరైన పరిష్కారం తప్పు సమస్యని పరిష్కరించగలదు. Domain పరిమితి trap ని 99.6% నుంచి 2.0% కి తగ్గించింది, మరియు coverage ఇప్పటికీ 2%. మెరుగుదల నిజమైనది, మరియు అది సరిపోలేదు.
6. లోతు పరిమితి trap రక్షణ కాదు. ఒక్క సంఖ్య కూడా మారలేదు – traps లోతైనవి కాదు, వెడల్పైనవి. ఆ పరిమితి ఇప్పటికీ ఉపయోగకరమే, వేరే కారణానికి.
7. గోడలు నష్టాన్ని పరిమితం చేస్తాయి; వాలులు ప్రవర్తనని మారుస్తాయి. $1/(1+n)$ అనే ఒక్క పదం – trap 0.1%, coverage 100%.
8. "ఏదీ తప్పు జరగలేదు" నియమాలు మళ్ళీ సరిపోలేదు. అయిదు నియమాలూ నెగ్గుతుండగా crawler 31.6% పేజీలను మౌనంగా పోగొట్టింది. దాన్ని పట్టుకునే సంఖ్య ఒక ఉల్లంఘన కాదు – ఒక ఖాళీ frontier.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Crawler ఒక fetching system కాదు – అది ఒక ఎంపిక చేసే system.

Web అనంతమైనది, మే బడ్జెట్ పరిమితమైనది. కాబట్టి ప్రతి design నిర్ణయం నిజంగా అడిగేది ఒకే ప్రశ్న: "ఈ తర్వాతి fetch దేని కోసం ఖర్చవుతోంది?"

§5 lo ఉజ్జాయింపు ఆ ఎంపికని మౌనంగా చేస్తుంది. §7 lo మర్యాద దాన్ని పరిమితం చేస్తుంది. §11 lo ప్రాధాన్యత దాన్ని స్పష్టంగా చేస్తుంది.

మరియు మీరు ఎంచుకోకపోతే – ఒక calendar మీ కోసం ఎంచుకుంటుంది.